Deep Learning Practical 3

Encoders, bottlenecks and latent distributions

1 Overview

We will train an MLP to predict colour from a greyscale photograph, using an encoder to compress it into a latent vector and a decoder to map that vector back to RGB. By comparing four and 64 latent values, passing image detail around the bottleneck through a skip connection, and interpolating between encoded images, we will examine what the latent vector retains. We will then inspect the distribution of these vectors and try generating images by sampling from a Gaussian.

Work through `dl-practical3.py` in IPython or VS Code, running its sections in order. Copy code from this file to preserve its formatting. Practical 4 extends this colourisation task to convolutional networks.

2 Colour prediction from a greyscale input

Let $x \sim p_d$ denote an RGB image drawn from the data distribution. Its greyscale version is $c = g(x)$, where

$$g(x) = 0.299R + 0.587G + 0.114B.$$

The network observes c and predicts x. CIFAR-10 images have 32×32 pixels, with values scaled to $[0, 1]$. We use a fixed split of 40,000 training and 10,000 validation images; the official test set is separate. The dataset is downloaded into `data` on first use and reused on later runs.

```
import torch
from torch import nn
from torch.nn import functional as F
from torchvision.datasets import CIFAR10
import matplotlib.pyplot as plt
```

```
def data(root='data', split='train'):
    if split not in ('train', 'validation'):
        raise ValueError('choose train or validation')
    dataset = CIFAR10(root, train=True, download=True)
    order = torch.randperm(50000, generator=torch.Generator().manual_seed(3801))
    index = order[:40000] if split == 'train' else order[40000:]
    rgb = torch.tensor(dataset.data[index], dtype=torch.float32).permute(0, 3, 1, 2)/255
    return rgb, torch.tensor(dataset.targets)[index]
```

```
images, labels = data()
validation, categories = data(split="validation")
```

The Python file includes the plotting functions used below, so you can run it on its own. The function `plot_rows` displays one labelled row per batch of images.

```
def grey(rgb):
    weights = rgb.new_tensor([.299, .587, .114])[None, :, None, None]
    return (rgb*weights).sum(1, keepdim=True)
```

```
mono = grey(images[:8])
print(images.shape, mono.shape, mono.flatten(1).shape)
plot_rows([mono.expand(-1, 3, -1, -1), images[:8]],
          ["Greyscale", "RGB target"])
plt.show()
```

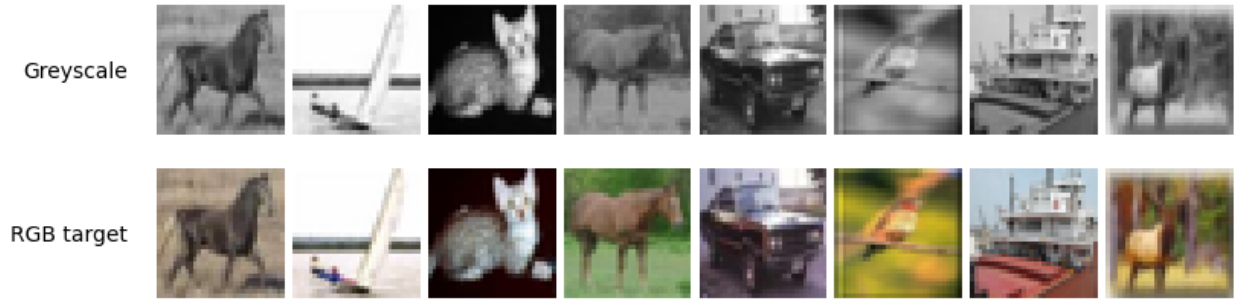


Figure 1: Each column pairs a greyscale training image with the RGB target we want the network to predict. To plot the two rows together, `expand` repeats each grey value across RGB, while the network itself still receives just one channel.

Exercise

1. How many pixel intensity values are in one 32×32 greyscale image? How many are in its RGB target?
2. Find two RGB colours with the same grey value. Start from $(0.5, 0.5, 0.5)$ and add $(0.0587, -0.0299, 0)$. What information has the conversion discarded?

3 Encode an image into a small vector

We represent the encoder by $z = E_\phi(c)$ and the decoder by $\hat{x} = D_\theta(z)$. These two MLPs communicate through the d values in z, which form the *bottleneck*. The decoder receives only this vector, so any information it needs about the input photograph must pass through those d numbers.

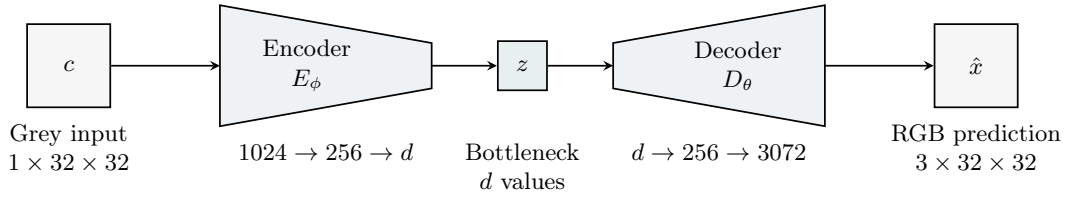


Figure 2: An encoder–decoder for colourisation, with image shapes written as channels × height × width (omitting the batch dimension). Layer widths appear below each network, and both use a ReLU after the hidden layer of width 256. As in Lecture 3's autoencoder, the input is compressed into a vector and then expanded into an image, but here the target is RGB and the input is grey.

Begin with `skip=False`, so `decode` simply returns the decoder's RGB output. We will use the optional addition later.

```
class Colouriser(nn.Module):
    def __init__(self, latent=64, skip=False, width=256):
        super().__init__()
        self.skip = skip
        self.encoder = nn.Sequential(nn.Flatten(), nn.Linear(1024, width), nn.ReLU(),
                                     nn.Linear(width, latent))
        self.decoder = nn.Sequential(nn.Linear(latent, width), nn.ReLU(),
                                     nn.Linear(width, 3072), nn.Unflatten(1, (3, 32, 32)))
        nn.init.zeros_(self.decoder[2].weight)
        nn.init.zeros_(self.decoder[2].bias)

    def decode(self, z, mono=None):
        rgb = self.decoder(z)
        return rgb + mono if self.skip else rgb

    def forward(self, mono):
        return self.decode(self.encoder(mono), mono)
```

```
model = Colouriser(latent=4)
z = model.encoder(mono)
rgb = model.decode(z)
print(z.shape, rgb.shape)
print(sum(p.numel() for p in model.parameters()))
```

The last affine layer starts at zero, and the other layers use PyTorch's random initialisation. Although predictions are clipped to $[0, 1]$ for display and evaluation, training uses the raw outputs so that gradients can move predictions back into range.

Define $\ell(\hat{x}, x)$ as the mean squared difference over an image's RGB values. The training objective is

$$\boxed{\mathcal{L}(\phi, \theta) = \mathbb{E}_{x \sim p_d}[\ell(D_\theta(E_\phi(g(x))), x)]\,.}$$

At each update we estimate this expectation using a minibatch, with both the encoder and decoder receiving gradients from the same loss.

Exercise

1. Trace x through g, E_ϕ and D_θ to $\hat{x}$. Which weights and biases are learned during training?
2. Count the encoder and decoder weights and biases for $d = 4$ and $d = 64$. Why does the parameter count grow much less than sixteenfold?
3. With `skip=False`, remove both ReLUs and write the network as $\hat{x} = Ac + b$. Express A and b using the layer weights and biases. How does d bound $\operatorname{rank}(A)$?

4 Train the bottlenecks and add a skip connection

Each model trains on the same sequence of minibatches. Selecting them with a separate random-number generator keeps the data order fixed when the layer sizes change.

```
def train(model, images, steps=6000, batch=64, device='cuda', learning_rate=.001, seed=3802):
    model.to(device).train()
    optimiser = torch.optim.Adam(model.parameters(), lr=learning_rate)
    batches = torch.Generator().manual_seed(seed)
    losses = []
    for step in range(steps):
        index = torch.randint(len(images), (batch,), generator=batches)
        target = images[index].to(device)
        loss = F.mse_loss(model(grey(target)), target)
        optimiser.zero_grad(set_to_none=True)
        loss.backward()
        optimiser.step()
        losses.append(loss.item())
    return losses
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
models, histories = {}, {}
for d in (4, 64):
    torch.manual_seed(3801)
    name = f"{d} values"
    net = Colouriser(latent=d)
    histories[name] = train(net, images, device=device)
    models[name] = net.eval()
```

Both runs use 6,000 updates, batch size 64 and learning rate 0.001. Compare how much detail each can reconstruct before adding a skip connection.

Colourisation is an *aligned image-to-image* task: each input pixel and its target refer to the same position in the same photograph. Because the grey input already contains edges and shading in the right places, we can carry these directly to the output. Write $r(c)$ for the grey image repeated across three channels, so it has the same shape as the decoder's RGB output. Adding the two gives

$$\hat{x} = r(c) + D_\theta(E_\phi(c)).$$

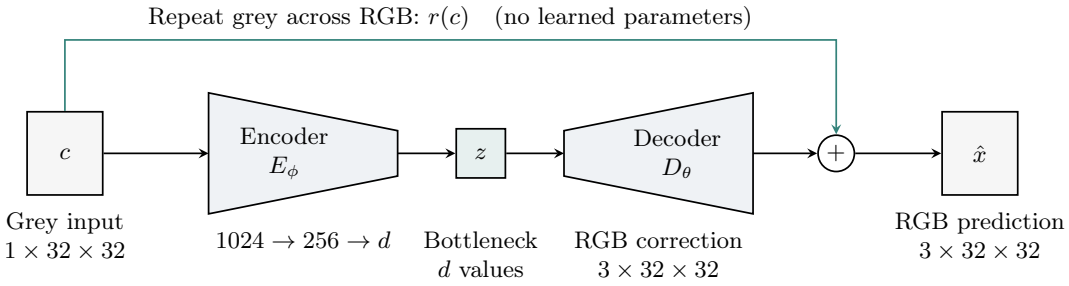


Figure 3: A fixed skip connection carries the grey pixels around the bottleneck. The decoder now predicts a correction, which is added pixel by pixel to the repeated-grey image. Unlike the U-Net in Lecture 3, this model has a single image-to-output skip, without connections between intermediate feature layers.

The loss still compares the *complete prediction* with the RGB target:

$$\mathcal{L}_{\text{skip}}(\phi, \theta) = \mathbb{E}_{x \sim p_d}[\ell(r(g(x)) + D_\theta(E_\phi(g(x))), x)] \,.$$

This route preserves all 1,024 greyscale values without adding parameters, leaving the decoder to learn a correction that adds colour and can also adjust brightness. Since the output layer starts at zero, the initial prediction is simply the repeated-grey image. In the code, adding `mono` to `rgb` uses PyTorch broadcasting to repeat its single channel across the three output channels.

```
for d in (4, 64):
    torch.manual_seed(3801)
    name = f"{d} values + skip"
    net = Colouriser(latent=d, skip=True)
    histories[name] = train(net, images, device=device)
    models[name] = net.eval()
with torch.no_grad():
    target = validation[:8].to(device)
    c = grey(target)
    rows = [c.expand(-1, 3, -1, -1)]
    rows += [net(c) for net in models.values()]
    rows += [target]
plot_rows(rows, ["Greyscale"] + list(models) + ["Target"])
plt.show()
```

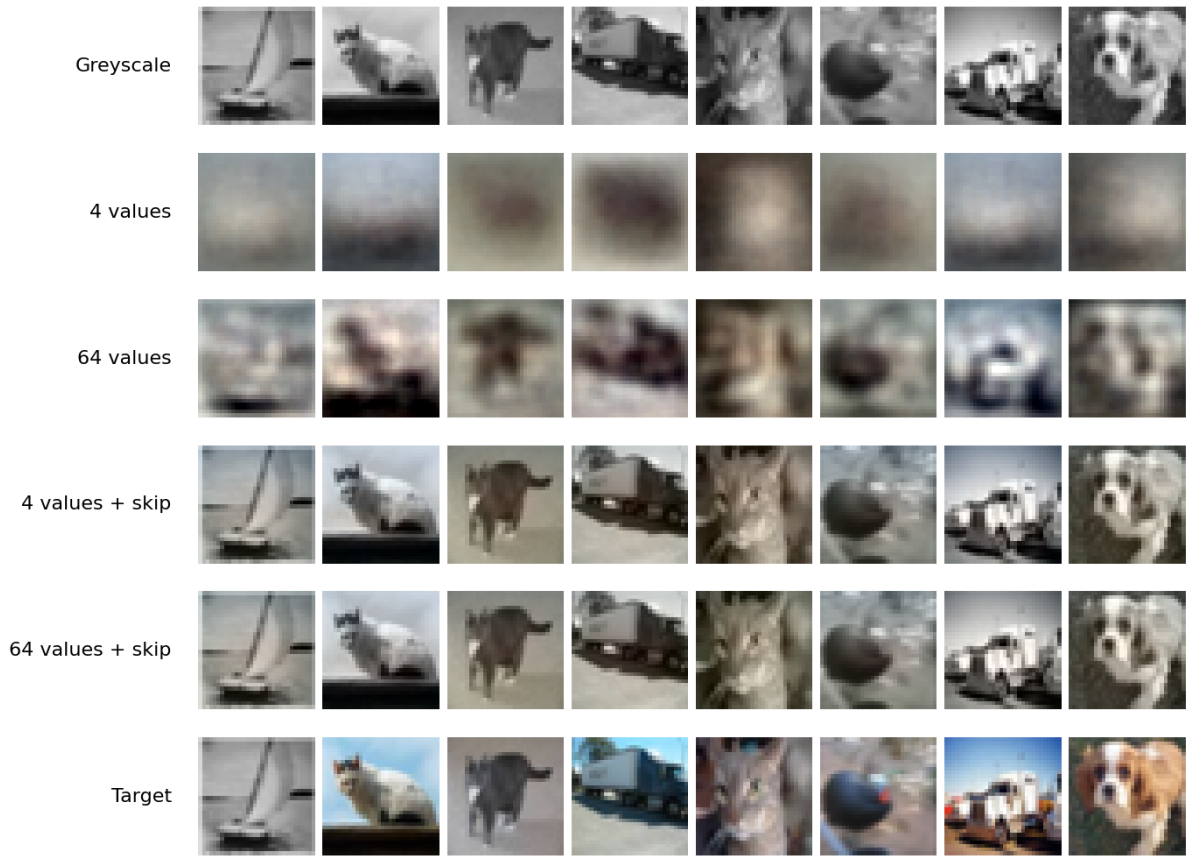


Figure 4: The first eight validation images. Four latent values retain broad regions of light and colour; 64 retain more structure. With a skip, image detail can bypass either bottleneck. Each model uses 6,000 updates.

Exercise

1. Compare the sail's edge, vehicle windows and animal faces. Why can the four-value skip model produce much more detail than the four-value plain model?
2. For two images, use the $d = 4$ skip model to zero their latent vectors, then swap their original vectors. Keep each grey input fixed. How do colour, brightness and outlines change?
3. Train plain and skip models with $d = 16$, keeping other settings fixed. Compare with $d = 4$ and $d = 64$. Does sharper output imply more shape information in the latent vector?

Training length and colour uncertainty

```
for name, net in models.items():
    print(name, evaluate(net, validation, device))
for name, history in histories.items():
    plt.semilogy(history, label=name)
plt.xlabel("Update"); plt.ylabel("Training RGB MSE")
plt.legend(); plt.show()
```

Measured validation RGB MSE for the same architectures and minibatch order:

Model	1,500 updates	6,000 updates	15,000 updates
4 values	0.031457	0.030590	0.030210
64 values	0.016326	0.013175	0.012929
4 values + skip	0.005847	0.005821	0.005796
64 values + skip	0.005890	0.005799	0.005775

Simply repeating the grey input across RGB gives validation MSE 0.006594. Even after training, both models without a skip have larger errors because they lose so much spatial detail through the bottleneck. The skip models preserve that detail and improve on the grey prediction by adding colour. Longer training helps the plain 64-value model substantially, while the skip models' improvements are smaller. All measurements use the same 10,000 validation images and seed 3801, although fresh runs can vary.

The ambiguity of colour prediction also limits what the model can recover: among all prediction functions, the ideal squared-error predictor is the conditional mean,

$$f^*(c) = \mathbb{E}_{p_d(x|c)}[x].$$

If several colours are plausible for the same grey input, their average can be muted. A larger model or longer run can bring the prediction closer to this average, but recovering the particular original colour may require information absent from c.

Exercise

Retrain one model with a different weight seed or more updates. Compare validation RGB MSE and colours. Then train with mean absolute error (L_1). For one pixel channel, what statistic of the target intensity given the grey input minimises expected absolute error?

5 Interpolate the two paths separately

Encode the second and third validation photographs and interpolate their latent vectors:

$$z_a = (1-a)z_0 + az_1, \qquad 0 \le a \le 1.$$

For the plain MLP, $D_\theta(z_a)$ determines the whole image. The skip model instead produces $r(c) + D_\theta(z_a)$, giving us two inputs to vary. First we blend both the latent code and the grey image, then keep the grey image fixed while blending only the code. Holding one path fixed lets us see what the other contributes:

```
plain = models["64 values"]
skipped = models["64 values + skip"]
with torch.no_grad():
    c = grey(validation[[1, 2]].to(device))
    a = torch.linspace(0, 1, 9, device=device)[:, None]
    z = plain.encoder(c)
    s = skipped.encoder(c)
    mixed_z = (1-a)*z[:1] + a*z[1:]
    mixed_s = (1-a)*s[:1] + a*s[1:]
    b = a[:, :, None, None]
    mixed_grey = (1-b)*c[:1] + b*c[1:]
    rows = [plain.decode(mixed_z),
            skipped.decode(mixed_s, mixed_grey),
            skipped.decode(mixed_s, c[:1]),
            skipped.decode(mixed_s, c[1:])]
plot_rows(rows, ["Latent only", "Blend both paths",
                      "Keep first grey", "Keep last grey"])
plt.show()
```

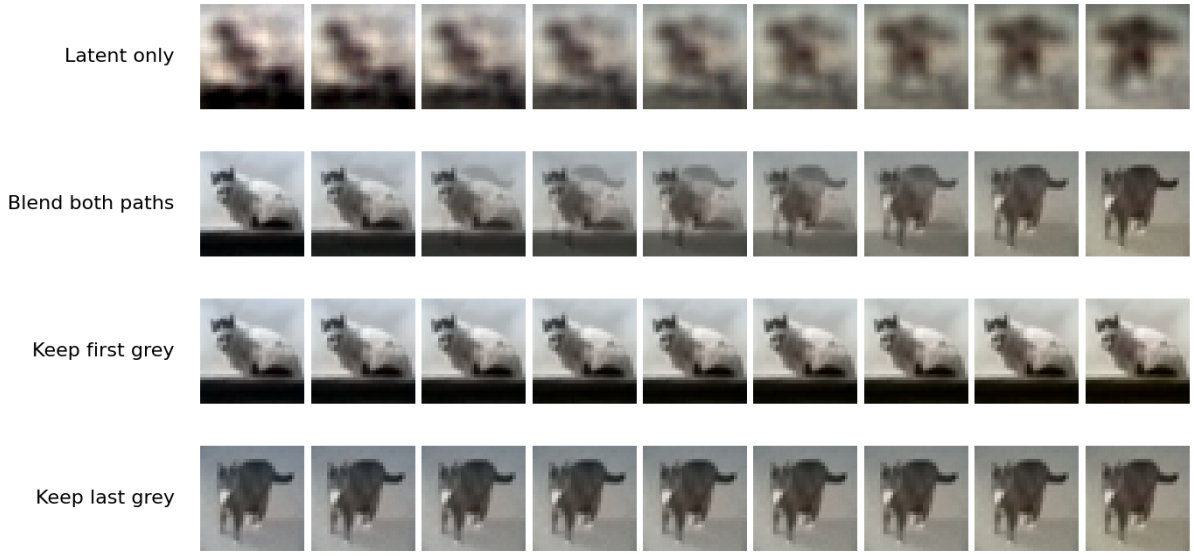


Figure 5: Left to right: $a = 0$ to $a = 1$. The top row uses the plain 64-value MLP. The remaining rows use the 64-value skip model with different inputs on its grey path.

Exercise

1. At $a = 1$, which rows match their model's prediction for the second image in the pair? What causes overlapping outlines in the "Blend both paths" row?
2. Compare $D_\theta((1-a)z_0 + az_1)$ with $(1-a)D_\theta(z_0) + aD_\theta(z_1)$ before clipping. Give a condition under which they agree for every $a \in [0, 1]$.
3. Repeat the interpolation plot with both $d = 4$ models. In each row, which changes come from the decoder and which from the grey skip input?

6 What distribution do the latent vectors follow?

Encoding different photographs gives different latent vectors. Imagine plotting one point per photograph in the d-dimensional latent space: the resulting cloud shows where the encoder places the data. Its distribution is called the *aggregate encoded distribution*.

For a fixed input c, the encoder always returns the same z, so its conditional distribution is concentrated at one point:

$$q_\phi(z \mid c) = \delta(z - E_\phi(c)) \,.$$

Here δ denotes a point mass. Averaging over photographs gives the distribution of the whole cloud:

$$\boxed{q_\phi(z) = \mathbb{E}_{x \sim p_d}[q_\phi(z \mid g(x))] \,.}$$

To estimate the shape of this distribution, encode training images with the four-value plain MLP, whose small bottleneck lets us inspect every coordinate. Each panel shows one coordinate across the same images, using shared axes and histogram bins so we can compare its mean, spread and shape with the others:

```
plain4 = models["4 values"]
with torch.no_grad():
    z = plain4.encoder(grey(images[:4096]).to(device)).cpu()
print(z.mean(0), z.std(0))
plot_marginals(z)
plt.show()
```

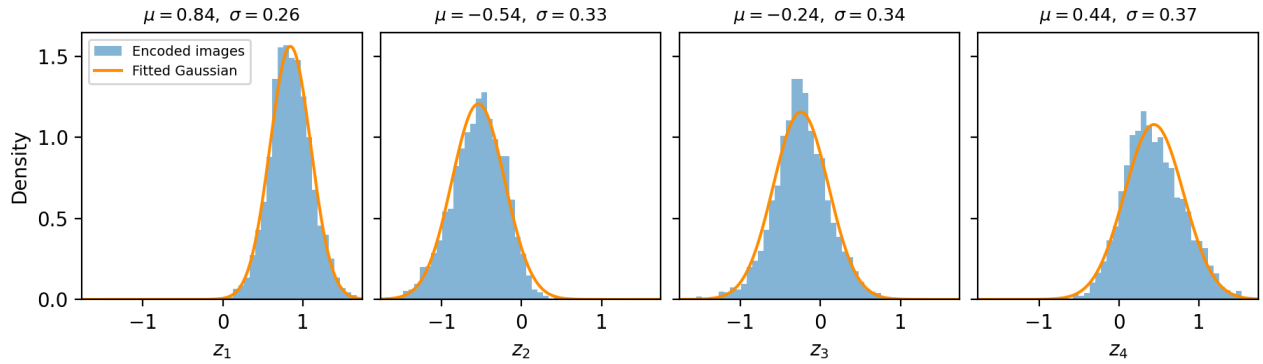


Figure 6: Four coordinates from the same trained encoder, evaluated on 4,096 training images. All panels use identical axis limits and histogram bins; their headings report the measured mean μ and standard deviation σ. Each curve is a Gaussian fitted to that coordinate. Similar bell shapes can have different locations and spreads, and Gaussian-looking marginals alone cannot establish the joint distribution.

Exercise

1. Plot the first two latent coordinates across training images. What suggests dependence? Can the plot establish independence? Would standardising each coordinate remove dependence?
2. Does the loss encourage $q_\phi(z)$ to match a Gaussian? What information does it encourage the codes to retain?

Encoded distributions and posteriors

In a probabilistic generative model, a prior $p(z)$ and likelihood $p_\theta(x \mid z)$ define a posterior $p_\theta(z \mid x)$ describing which codes are plausible after observing x. Lecture 8 develops the idea of training an encoder to approximate that posterior. Here, our encoder returns a single code for each grey input, and its aggregate distribution describes how these codes vary across the data. Interpreting it as a posterior approximation would require a generative model and a corresponding training objective.

7 Try a Gaussian in the latent space

So far, every decoded vector has come from an image or an interpolation between images. To generate without an input photograph, we need a way to choose new latent vectors. We will try fitting a Gaussian to the encoded training images and decoding samples from it:

$$z \sim \mathcal{N}(\hat{\mu}, \hat{\Sigma}), \qquad \hat{x} = D_\theta(z).$$

This fit is a separate step after training: the reconstruction loss did not require the encoded images to follow a Gaussian. Using the full covariance allows the fit to capture correlations between coordinates. To sample from it, we compute a Cholesky factor A with $AA^\top = \hat{\Sigma} + \varepsilon I$ and transform standard normal noise ϵ into $\hat{\mu} + A\epsilon$. The small diagonal addition makes the factorisation stable, and the code uses the equivalent row-vector calculation `noise @ scale.T`.

```
def gaussian_samples(values, count=8, seed=3803):
    mean = values.mean(0)
    centred = values - mean
    covariance = centred.T @ centred / (len(values)-1)
    eye = torch.eye(values.shape[1], device=values.device, dtype=values.dtype)
    scale = torch.linalg.cholesky(covariance + 1e-5*eye)
    rng = torch.Generator(device=values.device).manual_seed(seed)
    noise = torch.randn(count, values.shape[1], generator=rng,
                        device=values.device, dtype=values.dtype)
    return mean + noise @ scale.T
```

```
rows, names = [], []
with torch.no_grad():
    for d in (4, 64):
        net = models[f"{d} values"]
        z = net.encoder(grey(images[:4096]).to(device)).cpu()
        sampled = gaussian_samples(z).to(device)
        rows += [net(grey(validation[:8]).to(device)),
                 net.decode(sampled)]
        names += [f"{d}: encoded", f"{d}: Gaussian"]
plot_rows(rows, names)
plt.show()
```

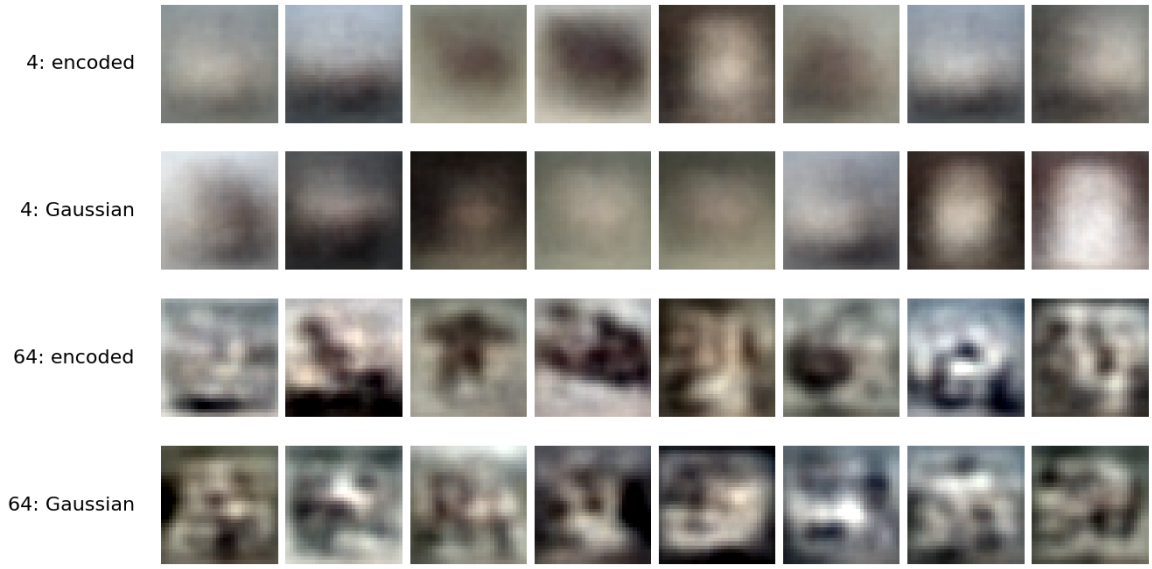


Figure 7: Predictions from encoded validation images, followed by Gaussian latent samples. The sampled rows have no paired target image. Four values produce smooth regions resembling the severely compressed reconstructions; the 64-value decoder can also produce more complicated, implausible arrangements.

Exercise

1. Compare Gaussian samples with encoded-image predictions for each d. For $d = 4$, does similarity in blur and colour imply realistic object structure?
2. Replace the fitted samples with `torch.randn(8, d)`. How might their means, variances and correlations mismatch the training codes? Do mean and covariance fully specify any distribution?
3. Decode eight training-image codes instead of Gaussian samples. Which model predictions does this reproduce? What assumption does Gaussian sampling add about the latent distribution?

8 What happens when we sample the skip path?

Sampling z alone is insufficient for the skip model, because its output also depends on a complete grey image c. Keeping a real photograph on that path would supply the object's shape instead of generating it. We will therefore try sampling both inputs by fitting a joint Gaussian to (z, c) using the four-value skip model. Each example contributes four latent values and 1,024 grey pixels, giving 1,028 values in total. Their covariance includes relationships between latent coordinates and pixels as well as between pixels:

```
skipped4 = models["4 values + skip"]
with torch.no_grad():
    c_train = grey(images[:4096])
    z = skipped4.encoder(c_train.to(device)).cpu()
    joint = torch.cat([z, c_train.flatten(1)], dim=1)
    sampled = gaussian_samples(joint)
    new_z = sampled[:, :4].to(device)
    new_grey = sampled[:, 4:].reshape(8, 1, 32, 32).to(device)
    real_grey = grey(validation[:8]).to(device)
    rows = [real_grey, skipped4.decode(new_z, real_grey),
            new_grey, skipped4.decode(new_z, new_grey)]
plot_rows(rows, ["Real grey", "Real grey + random z",
                 "Gaussian grey", "Both paths sampled"])
plt.show()
```

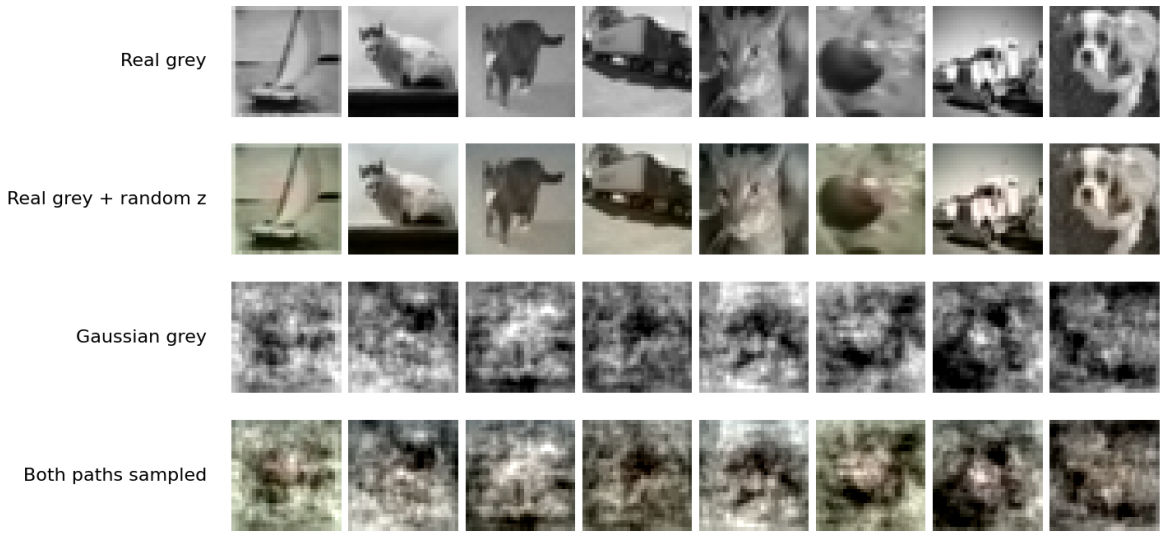


Figure 8: The real grey inputs provide recognisable objects even with random colour codes. Sampling both paths from the joint Gaussian produces unstructured textures. Matching pixel covariance retains some spatial smoothness, but loses the arrangements that make a photograph.

Exercise

1. In the second row, where does the recognisable object come from? Does a random z make this a sample of a new object?
2. The two sampled paths have a joint Gaussian distribution. Does that enforce the nonlinear relation $z = E_\phi(c)$ seen during training? Check the sampled z against `skipped4.encoder(new_grey)`.
3. Replace the joint Gaussian's covariance by its diagonal, keeping its mean fixed. How does this affect spatial structure in the sampled grey images and RGB outputs?
4. What extra distributions or training objectives would we need to generate photographs without supplying a real grey image? What would we need to generate several plausible colourisations of one fixed grey image?

The current colouriser predicts $\hat{x} = f(g(x))$ conditional on a photograph, so generating new images also requires a suitable distribution over decoder inputs. We will learn to model these distributions in the following practicals.

Further reading. Dive into Deep Learning, Section 5.1, *Multilayer Perceptrons*: `https://d2l.ai/chapter_multilayer-perceptrons/mlp.html`. For a later colourisation method that predicts a distribution over colours, see Zhang, Isola and Efros, *Colorful Image Colorization* (2016): `https://arxiv.org/abs/1603.08511`.